Assignment 2

----- Submitted by **Rikk Bera**

1. Self-assessment: LLM, Deep Learning, AI and ML

   I would currently place myself at level B in these areas cause my stream is AIML and also I did some projects on these topics but I don't have any professional experience on AIML. My final year project was on fake news detection using ML and NLP. In that project I used Roberta and rag but these are academic projects only. I have project-based experience with machine learning and deep learning, including model training and evaluation, computer vision. I understand the core concepts and have applied them in projects and I aim to do so in future.

Area	Rating
LLM (Large Language Models)	B
Deep Learning	B
Artificial Intelligence (AI)	B
Machine Learning (ML)	B

2. Key architectural components to create a chatbot based on LLM3

   A LLM based chatbot uses a large language model to understand the queries, context, and then generate human-like responses. Some of its prime example is Chatgpt or Gemini.

Now a LLM Chatbot consist of multiple architectural layers which goes like

User Interface → Backend API → Orchestration Layer → LLM → Response Processing → User Receives the Answer

A. **User Interface**: It's the frontend part which a user see and interact with. Its mainly take input from user and shows the output from the chatbot.
B. **Backend API** : The backend receives messages from the frontend and sends them to the appropriate services. It manages authentication, sessions, request validation, and communication with the LLM. Python frameworks such as Flask and FastAPI are commonly used to build this part.
C. **Orchestration layer :** This component coordinates the entire workflow. It constructs prompts, maintains conversation context, retrieves relevant information, and decides whether to call the LLM or an external tool. Frameworks such as LangChain and LlamaIndex can help implement these workflows.
D. **Large Language Model(LLM)** : The LLM is the core intelligence of the chatbot. It interprets the user's question and generates a relevant response. We can use a hosted model through an API or deploy an open-source model such as OLlama. We can also Fine-tune it to make it useful for specialized tasks, but it is not always necessary.
E. **Response Processing:** This is the main part for an Ai based chat bot and it consist of some critical components or steps.

   a. **Knowledge Base and Vector Database (RAG)** : So, if we want our chatbot to answer questions using company documents or private data, we can use Retrieval-

Augmented Generation (RAG). In this approach, we first divide the documents into smaller chunks and convert them into embeddings. These embeddings are then stored in a vector database. So, whenever a user asks a question, the system searches the database and retrieves the most relevant information. This information is then provided to the LLM as additional context, so the model can generate a more accurate and relevant answer. We can use tools such as FAISS, Chroma, or Pinecone for vector storage and similarity search. So, if the chatbot only needs to answer general questions, we may not need a vector database.

b. **Database and Conversation Memory:** In this part, a chatbot may also need a database to store user information, chat history, preferences, and other application data. We can use PostgreSQL or MongoDB to store this information, while Redis can be used for caching and fast access to session data. Conversation memory is also important because it helps the chatbot maintain context from previous messages. So, when a user asks a follow-up question, the chatbot can use the previous conversation to understand what the user is talking about instead of treating the question as a completely new one.

c. **Tools and External Integrations:** Sometimes the LLM alone is not enough to complete a user's request. So in such cases, we can connect the chatbot to external APIs, databases, or other tools. For example, the chatbot can retrieve order details, check weather information, search a database, or create a support ticket. So, the LLM can identify when an external tool is needed, and the backend can execute the required function and return the result. This makes the chatbot more useful because it can perform specific tasks instead of only generating text. However, we must validate tool requests and control access to external systems to prevent unauthorized actions.

d. Security, Monitoring, and Evaluation: Security is also an important part of building a reliable chatbot. We need to validate user inputs, protect sensitive information, store API keys securely, and implement proper authentication and authorization. We should also monitor the chatbot after deployment to check its performance. For example, we can track response time, token usage, API costs, errors, and response quality. So, by using logs and evaluation tests, we can identify problems, check whether the chatbot provides accurate answers, and improve its performance over time.

F. **User Receive the answer:** Finally, after the response has been generated and processed, the backend sends the final answer to the frontend. So, the user can see the response in the chat interface. If the user asks another question, the same process continues, and the chatbot uses the available conversation context to respond accordingly.

3. **vector databases:**

A vector database is a type of database that stores data in the form of vectors, which are numerical representations of data such as text, images, and documents. These numerical representations are called embeddings. So, the main purpose of a vector database is to find information based on its meaning and similarity rather than only matching exact words.

For example, suppose we are building an AI chatbot that answers questions about a company's policies. A user asks, "How many days can I take leave?" But the company document says, "Employees are allowed 20 days of annual leave." So, even though the words in the question and the document are different, their meanings are similar. A vector search system can identify this similarity and retrieve the relevant information.

The process starts by dividing documents into smaller chunks. So, each chunk is converted into an embedding using an embedding model, and these embeddings are

stored for searching. When a user asks a question, the system also converts the question into an embedding and compares it with the stored embeddings. So, the system retrieves the most relevant chunks and sends them to the LLM as context. This process is commonly used in Retrieval-Augmented Generation (RAG), which helps the chatbot generate answers using relevant information from external documents.

Some popular tools for vector search include FAISS, Chroma, Pinecone, and Weaviate. So, the choice depends on the project's requirements, such as the amount of data, performance, cost, and scalability.

Reason for choosing a Vector Database, and Why?

Suppose I am building an AI-powered customer support chatbot for an e-commerce company. The chatbot should answer questions about product details, shipping, refunds, return policies, and other frequently asked questions. The company has thousands of documents, so the chatbot needs a way to search those documents and find the correct information whenever a customer asks a question.

For this project, I would choose **FAISS (Facebook AI Similarity Search)** for vector search.

I would choose FAISS because it is an efficient library for storing and searching high-dimensional vectors. It supports similarity search, works well with Python, and can handle a large number of embeddings. So, it is a good choice when we want to build a fast and cost-effective RAG-based chatbot without depending on a separate managed vector database service. FAISS runs locally, which also gives us more control over our data and infrastructure.

The implementation would start by collecting the company's FAQs, product information, and return policies. So, I would divide these documents into smaller chunks and use an embedding model to convert each chunk into a numerical vector. These vectors would then be stored in a FAISS index, while the original text and other required information would be stored separately and mapped to their corresponding vectors.

Whenever a customer asks a question, the system converts that question into an embedding and searches the FAISS index for the most similar vectors. So, the system retrieves the corresponding document chunks and passes them to the LLM as context. The LLM then uses this information to generate a relevant answer, which is finally displayed to the customer.

For example, if a customer asks, "Can I return a product after one week?", the chatbot searches the stored document embeddings and retrieves the company's return policy. So, the LLM can generate an answer based on that policy instead of providing a general or unsupported response.

One important thing we should remember is that FAISS is a vector search library, not a complete database system. So, it does not automatically manage document storage, user information, or chat history. We may need a separate database to store the original documents, metadata, and other application data.

Overall, I would choose FAISS because it provides fast similarity search, integrates easily with Python-based machine learning applications, and does not require a paid managed service. So, for this hypothetical customer support chatbot, FAISS would be a practical choice for implementing the vector search component of the RAG architecture.